The `getting_pin` state function is a bit more complex in that it can handle three distinct types of message, as in figure 4.3. This is shown in the following listing.

Listing 4.16. The `getting_pin` state function for the simple ATM implementation

```
void atm::getting_pin()
{
    incoming.wait()
        .handle<digit_pressed>(                    <-(1)
            [&](digit_pressed const& msg)
            {
                unsigned const pin_length=4;
                pin+=msg.digit;
                if(pin.length()==pin_length)
                {

                    bank.send(verify_pin(account,pin,incoming));
                    state=&atm::verifying_pin;
                }
            }
            )
        .handle<clear_last_pressed>(               <-(2)
            [&](clear_last_pressed const& msg)
            {
                if(!pin.empty())
                {
                    pin.resize(pin.length()-1);
                }
            }
            )
        .handle<cancel_pressed>(                   <-(3)
            [&](cancel_pressed const& msg)
            {
                state=&atm::done_processing;
            }
            );
}
```

This time, there are three message types you can process, so the `wait()` function has three `handle()` calls chained on the end (1), (2), (3). Each call to `handle()` specifies the message type as the template parameter and then passes in a lambda function that takes that particular message type as a parameter. Because the calls are chained together in this way, the `wait()` implementation knows that it's waiting for a `digit_pressed` message, a `clear_last_pressed` message, or a `cancel_pressed` message. Messages of any other type are again discarded.

This time, you don't necessarily change state when you get a message. For example, if you get a `digit_pressed` message, you just add it to the `pin` unless it's the final digit. The main loop (7) in listing 4.15) will then call `getting_pin()` again to wait for the next digit (or clear or cancel).

This corresponds to the behavior shown in figure 4.3. Each state box is implemented by a distinct member function, which waits for the relevant messages and updates the state as appropriate.

As you can see, this style of programming can greatly simplify the task of designing a concurrent system, because each thread can be treated entirely independently. It is thus an example of using